```java
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

  // Mapper Class
  public static class TokenizerMapper
      extends Mapper<Object, Text, Text, IntWritable> {

    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    public void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {

      StringTokenizer itr = new StringTokenizer(value.toString());

      while (itr.hasMoreTokens()) {
        word.set(itr.nextToken());
        context.write(word, one);
      }
    }
  }

  // Reducer Class
  public static class IntSumReducer
      extends Reducer<Text, IntWritable, Text, IntWritable> {

    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values,
              Context context)
        throws IOException, InterruptedException {

      int sum = 0;
```

```
            for (IntWritable val : values) {
                sum += val.get();
            }

            result.set(sum);
            context.write(key, result);
        }
    }

    // Driver Class
    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "word count");

        job.setJarByClass(WordCount.class);

        job.setMapperClass(TokenizerMapper.class);
        job.setCombinerClass(IntSumReducer.class);
        job.setReducerClass(IntSumReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Input:

Hello World Hello Hadoop

```
C:\WordCount>javac -classpath %HADOOP_HOME%\share\hadoop\common\*;%HADOOP_HOME%\share\hadoop\mapreduce\* -d . WordCount.java

C:\WordCount>jar -cvf wordcount.jar *
added manifest
adding: input.txt(in = 27) (out= 25)(deflated 7%)
adding: output/(in = 0) (out= 0)(stored 0%)
adding: output/.part-r-00000.crc(in = 12) (out= 14)(deflated -16%)
adding: output/._SUCCESS.crc(in = 8) (out= 10)(deflated -25%)
adding: output/part-r-00000(in = 25) (out= 25)(deflated 0%)
adding: output/_SUCCESS(in = 0) (out= 0)(stored 0%)
adding: WordCount$IntSumReducer.class(in = 1739) (out= 739)(deflated 57%)
adding: WordCount$TokenizerMapper.class(in = 1736) (out= 753)(deflated 56%)
adding: WordCount.class(in = 1491) (out= 814)(deflated 45%)
adding: WordCount.java(in = 2401) (out= 745)(deflated 68%)

C:\WordCount>hadoop jar wordcount.jar WordCount input.txt output
Terminate batch job (Y/N)? n

C:\WordCount>hadoop jar wordcount.jar WordCount input.txt output
2026-04-15 09:19:36,301 INFO impl.MetricsConfig: Loaded properties from hadoop-metrics2.properties
2026-04-15 09:19:36,385 INFO impl.MetricsSystemImpl: Scheduled Metric snapshot period at 10 second(s).
2026-04-15 09:19:36,385 INFO impl.MetricsSystemImpl: JobTracker metrics system started
2026-04-15 09:19:36,856 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-04-15 09:19:36,942 INFO input.FileInputFormat: Total input files to process : 1
2026-04-15 09:19:36,981 INFO mapreduce.JobSubmitter: number of splits:1
2026-04-15 09:19:37,096 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_local1985344993_0001
2026-04-15 09:19:37,096 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-04-15 09:19:37,218 INFO mapreduce.Job: The url to track the job: http://localhost:8080/
```

```
                Combine input records=4
                Combine output records=3
                Reduce input groups=3
                Reduce shuffle bytes=43
                Reduce input records=3
                Reduce output records=3
                Spilled Records=6
                Shuffled Maps =1
                Failed Shuffles=0
                Merged Map outputs=1
                GC time elapsed (ms)=49
                Total committed heap usage (bytes)=429391872
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=27
        File Output Format Counters
                Bytes Written=37

C:\WordCount>type output\part-r-00000
hadoop  1
hello   2
world   1

C:\WordCount>
```